URBANVERSE

MEMBER 1 | BACKEND + INTEGRATION

Project: City-Scale Digital Twin for Multi-Hazard Prediction and Urban Resilience Planning

Hackathon target: Build a convincing, working flood-risk MVP for a small pilot area in Mangalore. Use real map/geographical data where practical and clearly label simulated rainfall and risk outputs.

Your mission

Own the backend, connect the team's modules, and keep one integrated version running.

Your tasks

- Set up the FastAPI backend inside backend/.
- Create POST /api/simulate using the shared request/response contract in the Team Integration Guide.
- Create GET /api/health so the frontend can check whether the backend is running.
- Call Member 4's simulation function; do not duplicate the risk formula in the API route.
- Load or read the pilot-area GeoJSON supplied by Member 3, and verify that returned zone IDs match it.
- Coordinate integration and review pull requests before merging into main.
- Write simple run instructions in README.md.

What you must hand over

- A backend that starts locally.
- GET /api/health returns a clear success response.
- POST /api/simulate accepts rainfall_mm and area_id and returns the agreed JSON shape.
- A tested example request and response in docs/API.md.
- A working integrated branch on GitHub.

File ownership and coordination

- Own backend/ and integration documentation.
- Do not change the frontend's API field names without telling Member 2.
- Do not invent GIS zone IDs; use Member 3's IDs.
- Do not rewrite Member 4's simulation formula without discussing it.

Start in this order

1. Create the backend folder and a minimal FastAPI app.
2. Add /api/health.
3. Create a temporary mock simulation response matching the contract.
4. Share the endpoint and example response with Member 2 immediately.
5. Replace mock output with Member 4's real function when it is ready.

Your acceptance test

Change rainfall_mm from 30 to 100 and confirm the API returns valid JSON with zone IDs, 0–100 scores, risk labels and simulation_only=true.

Team rule

Commit working changes to your own Git branch, push them, and tell the integration lead what changed. Do not edit another member's files without agreement. If you are blocked, share the exact error and the smallest reproducible example rather than silently changing the shared interface.